Université de Rennes

UFR mathématiques

C++ – Bases

8 octobre 2026

Support de cours

Master 2 Calcul Scientifique et Modélisation

Master 2 Simulation Numérique pour la Santé

Master 2 Cryptographie

par

Éric DARRIGRAND

Table des matières

Introduction **5**

1 Introduction au C++ **7**
1.1 Introduction . . . 7
1.2 Un langage compilé . . . 7
1.3 Organisation d'un code – programmation objet . . . 7
1.4 Évocation de la STL . . . 8
1.5 Aide . . . 9

2 Quelques fonctionnalités générales **11**
2.1 Introduction . . . 11
2.2 Structuration de la programmation dans des fichiers séparés . . . 11
2.3 Préprocesseur – inclusion de fichiers . . . 11
2.4 Entrée/sortie de base . . . 12
2.5 Entrée/sortie dans un fichier . . . 13
2.6 Fonction et arguments d'entrée avec valeur par défaut . . . 13
2.7 Surcharge de fonction . . . 13
2.8 Transmission par valeur, par pointeur ou par référence . . . 14
2.9 Surcharge d'opérateur . . . 14
2.10 Référence et mot clé `const` . . . 15
2.11 Fonction `assert` . . . 15
2.12 Allocation dynamique . . . 16
2.13 Fonctions génériques . . . 16

3 Les classes **19**
3.1 Introduction . . . 19
3.2 Définition d'une classe . . . 19
3.3 Constructeurs, destructeur et opérateurs d'une classe . . . 21
3.4 Mot clé `friend` et alternative . . . 24
3.5 Classes et allocation dynamique . . . 26
3.6 Classe générique . . . 29
3.7 Classe base et classe dérivée . . . 30

Bibliography **35**

Introduction

Ce document est un support de cours pour le module de C++ bases du M2 Math CSM/SNS/Crypto. Il présente quelques notions fondamentales du C++ en supposant connues les notions de base du C. Ce module vient avant le module de C++ compléments du M2 Math CSM/SNS/Crypto. De ce fait, certains éléments fondamentaux ne sont pas abordés ici mais dans le module suivant.

Le premier chapitre de ce document introduit le C++, le deuxième donne quelques notions générales et le troisième est consacré aux classes. Le cours aborde les notions dans un ordre différent. Notamment, la notion de classe est abordée dès les premières séances et est enrichie à plusieurs reprises au fil des séances.

Chapitre 1

Introduction au C++

1.1 Introduction

Le C++ est un langage compilé initié en 1979 par Bjarne Stroustrup. Les normalisations C++98 et C++11 ont marqué l'histoire du développement du C++. Depuis 2011, une nouvelle norme paraît tous les trois ans mais n'induit pas nécessairement des changements aussi conséquents. La norme C++11 a induit quant à elle d'importantes évolutions qui font qu'il y a véritablement un avant et un après 2011. Ceci implique l'importance de prendre en considération les avancées 2011 et de connaître le C++98. En effet, de nombreux codes ont été écrits selon la norme 98 et ne seront pas récrits, le C++11 étant compatible avec le C++98.

1.2 Un langage compilé

Le C++ est un langage compilé, tout comme le C ou le Fortran, et contrairement à Matlab ou Python qui sont des langages interprétés. Cela induit une structuration du code en fichiers de programmation, .c++, et en fichiers "header" de déclaration, .h. Les fichiers de programmation .c++ sont compilés possiblement un à un. Puis l'édition de lien permet de créer l'exécutable.

Sur les machines de l'université, vous utiliserez le compilateur `g++` avec les options

- `-c` : pour compiler exclusivement,
- `-std=c++11` : pour compiler et créer l'exécutable en C++ 2011,
- `-Wall` : pour afficher tous les warnings à la compilation,
- `-fsanitize=address` : pour compiler et créer l'exécutable avec l'option de contrôle des dépassements de mémoire. Cette option est couteuse. À utiliser en phase de débug.

1.3 Organisation d'un code – programmation objet

L'organisation d'un code passe par la considération de différents ingrédients :

- types et variables,
- fonctions,
- classes : permet de définir des types utilisateur,
- espaces de noms (`namespace`),
- ...

Ces objets permettent de structurer le code et de l'organiser. Les classes constituent un ingrédient clé du C++ qui permet l'orientation objet de la programmation. En langage C ou Fortran, la notion de classe n'existe pas. Ces langages de programmation "traditionnelle" ne permettent pas une programmation orientée objet. Le C++ a été créé avec l'ambition de permettre la programmation objet. Dans la programmation "traditionnelle", la modularité est basée sur la décomposition des traitements (fonctions, sous-fontions) que l'on applique à des données. Dans la programmation objet, la structure du programme est guidée par le type de données. L'analyse du problème conduit à définir des objets auxquels sont généralement associées des valeurs représentatives de l'état courant, ainsi que des actions qui traduisent ce qu'il est possible de faire avec ces objets.

1.4 Évocation de la STL

Le C++ intègre un certain nombre d'outils, des fonctions, des classes, ... bien utiles et disponibles pour l'utilisateur dans la bibliothèque standard, la "STL" ou "Standard Library". Ces outils sont essentiellement déclarés dans un ensemble de "headers", et font partie de l'espace de noms `std`.

Entre autres, cette bibliothèque contient :

- ⋆ des fonctions d'écriture et de lecture disponibles en référençant le header `<iostream>` par la directive

```
#include <iostream>
```

- ⋆ des fonctions mathématiques déclarées dans les headers `<cmath>` et `<cstdlib>`. Par exemple pour utiliser la fonction `sqrt`, l'utilisateur doit référencer le header `<cmath>` par la directive

```
#include <cmath>
```

- ⋆ des conteneurs ("containers") tel que `vector` ou `map`,
- ⋆ et bien d'autres outils (cf [Str14] p 859).

Pour utiliser la fonction `sqrt`, l'utilisateur doit spécifier l'espace de noms `std`. Pour cela, il écrira l'une des lignes suivantes :

- ⋆ En début des fichiers de programmation .c++, .cpp, pour permettre l'usage des noms des fonctions de la STL sans spécifier l'espace de noms au moment de leur usage, mais aussi imposer tous les noms contenus dans l'espace de noms `std` :

```
using namespace std;
```

- ⋆ À privilégier dans les fichiers de programmation .c++, .cpp, en amont de l'usage de la fonction, pour permettre l'usage de la fonction `sqrt` sans spécifier l'espace de noms au moment de son usage :

```
using std::sqrt;
```

- ⋆ À chaque appel de la fonction `sqrt` si on n'a pas choisi les options précédentes (c'est le cas dans les headers .h, .h++, .hpp) :

```
... std::sqrt(...) ...;
```

Pour la fonction `abs`, vous prendrez garde au fait que, sans mention de l'espace de noms `std`, le compilateur considèrera la fonction C qui n'aura pas le comportement attendu puisqu'il s'agira alors d'une fonction de type entier.

1.5 Aide

En complément de ce document, le lecteur peut se référer à l'ouvrage de Stroustrup [Str14] ou à d'autres ressouces en ligne :

- ⋆ le site du concepteur du C++ : https://www.stroustrup.com/
- ⋆ deux références correspondant au contenu (en libre accès) du livre de Bruce Eckel "Thinking In C++" : https://bruce-eckel.developpez.com/livres/cpp/ticpp/v1/ https://bruce-eckel.developpez.com/livres/cpp/ticpp/v2/
- ⋆ deux sites d'aide très utiles à l'usage : https://www.cplusplus.com/reference/ https://www.cppreference.com/
- ⋆ un autre site de référence concernant la STL : https://docs.microsoft.com/en-us/cpp/cpp/

Chapitre 2

Quelques fonctionnalités générales

2.1 Introduction

Cette section présente des notions générales largement utilisées. Il y est fait référence parfois à la notion de classe mais cette notion est présentée en détail dans le chapitre suivant.

Attention : tout au long de ce document, vous prendrez garde à la différence entre une déclaration et une définition (cf cours de programmation scientifique du M1CSM).

2.2 Structuration de la programmation dans des fichiers séparés

Un programme en C++ s'écrit dans des fichiers de deux types, les fichiers "header" qui contiennent usuellement des déclarations ou définitions de classes ou des déclarations de fonctions, et des fichiers de programmation qui contiennent les définitions des fonctions. Les fichiers "header" sont d'extension .h, .h++ ou .hpp. Dans ce cours, nous utiliserons l'extension .h. Pour les fichiers de programmation, nous choisissons l'extension .c++ (.cpp serait aussi possible).

La définition d'une classe se fait généralement au travers de deux fichiers, le header d'extension .h qui contient la définition de la classe avec l'identification des données membres et la déclaration des fonctions membres (+ la déclaration d'éventuelles fonctions externes relatives), puis un fichier d'extension .c++ qui contient la définition des fonctions membres (+ la définition des fonctions externes associées).

La répartition des différents composants d'un programme au travers des différents fichiers s'effectue de sorte que chaque fichier .c++ puisse être compilé séparément.

2.3 Préprocesseur – inclusion de fichiers

Lorsque des fonctions ou des classes sont déclarées dans un header, pour les utiliser dans un fichier de programmation, d'extension .c++, il faut référencer le header par la directive `#include` du préprocesseur. Par exemple, si une classe est définie dans le fichier `MaClasse.h` et que les fonctions associées sont programmées dans `MaClasse.c++`, ce dernier contiendra la ligne

```
#include "MaClasse.h"
```

en début de fichier.

D'autres `#include` peuvent intervenir, notamment pour référencer des headers de la STL. Il convient alors de choisir un ordre pour les headers. Voici un exemple d'organisation des headers telle que choisie dans la bibliothèque numérique FastMMLib [DLss], dans le fichier `FieldContribution.cpp` :

```
#include "FieldContribution.hpp"
#include <iostream>
using std::cout;
using std::endl;
```

Selon l'organisation du programme, selon sa complexité, il est possible qu'un header soit référencé plusieurs fois dans une même unité du programme. Cela provoquerait alors une erreur à cause de déclarations multiples. Pour éviter cela, il suffit d'utiliser des directives du préprocesseur qui assurent le parcours d'un header seulement s'il n'a pas déjà été considéré. Le contenu du fichier `MaClasse.h` pourrait être :

```
#ifndef MACLASSE_H
#define MACLASSE_H

/* Definition de la classe MaClasse. */

#endif  // MACLASSE_H
```

Un header peut contenir entre autres : définitions de types, déclarations de fonctions, définitions de classes, déclarations de variables globales, définitions de variables constantes, définitions de fonctions `inline`. Par contre, un fichier header ne peut pas contenir : définition ordinaire d'une fonction, définition d'une variable, usage de la directive `using`. Pour une liste plus exhaustive de ce qui peut être contenu ou pas dans un header, nous renvoyons à [Str14] p424.

Cette énumération requiert la définition d'une fonction `inline`. Une fonction définie dans un header doit être `inline`. Si elle est définie au sein de la définition d'une classe, alors, elle est `inline` par défaut. Par contre, si elle est définie dans un header en dehors d'une classe, elle doit être identifiée par le mot clé `inline`. Une fonction `inline` définie dans un header est prise en compte par le compilateur. Son appel est remplacé par l'action de la fonction à la compilation. Cela est à éviter pour les fonctions de grande taille. Généralement, cela est réservé aux fonctions qui tiennent sur une ligne. A contrario, une fonction définie dans le .c++ est prise en compte à l'édition de lien. Cela est à privilégier.

2.4 Entrée/sortie de base

Les entrées sorties en C++ se font par les flots ("stream") `cin` et `cout` à l'aide des opérateurs `>>` et `<<`. Ils nécessitent de référencer le header `<iostream>`. Usage :

```
#include <iostream>
using namespace std;
int main()
{
   int num_annee;
   cout << "Donner␣le␣numero␣de␣l'annee"<<endl;
   cin >> num_annee;
   cout << "Bonjour␣et␣bonne␣rentree␣"<<num_annee<<endl;
}
```

2.5 Entrée/sortie dans un fichier

Pour lire ou écrire dans un fichier, l'utilisateur définit des flots d'entrée et de sortie. Par exemple, nous pouvons considérer le programme suivant :

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;
int main () {
   int donnee1, donnee2;
   string ficdon("monfichier.don"), ficres("monfichier.res");
   // Ouverture du flot d'entree
   ifstream fdon(ficdon.c_str());
   if (! fdon.is_open()) {
      cout << "Fichier " << ficdon << " inexistant. Abandon." << endl;
      exit(1);
   }
   // Lecture des donnees selon le format du fichier
   fdon >> donnee1;
   fdon >> donnee2;
   // Fermeture du flot d'entree
   fdon.close();
   // Ouverture du flot de sortie
   ofstream fres(ficres.c_str());
   // Ecriture des resultats
   fres << "Les resultats sont" << endl;
   ...
   // Fermeture du flot de sortie
   fres.close();
}
```

Lors de la définition d'un flot d'entrée, il est vivement conseillé de vérifier que le fichier existe bien, par exemple à l'aide de la fonction membre `is_open`. Par ailleurs, les flots d'entrée ou de sortie sont fermés par appel de la fonction membre `close`.

2.6 Fonction et arguments d'entrée avec valeur par défaut

Une fonction peut admettre des arguments avec valeur par défaut, en fin de liste. Par exemple, la fonction

```
int som(int a, int b=1){return a+b;}
```

peut s'appeler avec un ou deux arguments d'entrée. Si elle est appelée avec un argument d'entrée, la valeur 1 sera attribuée au deuxième argument. De même, la fonction

```
int som(int a=0, int b=1){return a+b;}
```

peut s'appeler avec zéro, un ou deux arguments d'entrée.

2.7 Surcharge de fonction

En C++, la notion de surcharge est le fait que le développeur puisse donner le même nom à plusieurs fonctions dès lors que la liste des types des arguments d'entrée diffère. Pour le compilateur, le nom effectif d'une fonction prend en compte les types des arguments d'entrée.

Ainsi, les fonctions suivantes peuvent co-exister :

```
int add(int a, int b){return a+b;}
double add(double a, double b){return a+b;}
```

La notion de surcharge d'une fonction s'étend aussi aux fonctions membres d'une classe. Ainsi, lorsque la notion de classe sera abordée et la notion de constructeur d'une classe aussi, il sera rapidement question de définir plusieurs constructeurs portant tous le même nom. Il s'agira à nouveau de surcharge de fonction.

2.8 Transmission par valeur, par pointeur ou par référence

En C, il est possible de transmettre des variables par valeur ou par pointeur. Le C++ offre la possibilité de transmettre une variable par référence.

Cela nécessite d'introduire d'abord la notion de référence. Une référence est un alias vers un objet. Elle diffère d'un pointeur par les points suivants :

- ⋆ l'accès à la référence se fait avec la même syntaxe que l'accès à l'objet ;
- ⋆ une référence réfère toujours à l'objet auquel elle a été initialisée ;
- ⋆ il n'y a pas de référence "nulle", et il est supposé qu'une référence réfère toujours à un objet.

L'exemple suivant illustre cela :

```
int var = 1;
int& r (var);  // r et var referent au meme int
int x = r;     // x devient 1
r = 2;         // var devient 2
r++;           // var devient 3
int* p = &r;   // p pointe sur var
```

L'usage le plus courant des références est pour la transmission d'arguments d'entrée ou pour les valeurs retournées de fonctions ou opérateurs. Voici un exemple de fonctions comparables :

```
int add1(int a, int b){return a+b;}
int add2(int* a, int* b){return *a+*b;}
int add3(int& a, int& b){return a+b;}
```

L'usage de ces fonctions se fait ainsi :

```
int i = 1, j = 2;
int k = add1(i,j);
int l = add2(&i,&j);
int m = add3(i,j);
```

2.9 Surcharge d'opérateur

Tout comme les fonctions, il est possible de surcharger les opérateurs. Ainsi le développeur sera amené à définir des opérateurs comme +, += ou = pour des objets de type défini par ses soins.

La surcharge d'opérateur vérifie les restrictions suivantes :

- ⋆ L'un au moins des opérandes de l'opérateur surchargé doit être un objet (impossible de redéfinir les opérateurs prédéfinis du C++).
- ⋆ On ne peut pas créer un nouvel opérateur.

- ⋆ Un opérateur surchargé doit avoir le même nombre d'opérandes que l'opérateur d'origine (respect de la sémantique).
- ⋆ Les opérateurs ".", "::", "?:", "sizeof", ".⋆", ne peuvent pas être surchargés.
- ⋆ Les opérateurs "=", "()", "[]", et "->", ne peuvent être redéfinis que comme fonction membre d'une classe et non comme fonction externe.

En dehors d'une classe, nous trouvons des surcharges du type :

```
bool operator<(Type obj1, Type obj2){do something, return a boolean}
Type& operator+=(Type& obj1, Type obj2){do something, return a Type& object}
Type operator+(Type obj1, Type obj2){do something, return a Type object}
```

De manière générale, la surcharge d'opérateur comme fonction externe se fait selon les prototypes suivants :

- ⋆ pour un opérateur unaire :

```
type operator op(type)
```

- ⋆ pour un opérateur binaire :

```
type1 operator op(type2, type3)
```

Pour les surcharges d'opérateurs au sein d'une classe, nous renvoyons au chapitre 3.

2.10 Référence et mot clé const

La transmission d'une variable par référence à une fonction rend possible de modifier cette variable au sein de la fonction. Souvent, la transmission par référence est choisie afin d'éviter une recopie d'objet, sans intention de modifier l'objet. Dans ce cas, il est utile de protéger la variable associée. Cela se fait à l'aide du mot clé const.

Le prototype

```
int f(const int& a){...}
```

définit une fonction qui ne peut modifier la valeur de la variable a.

La fonction add3 de la section 2.8 se récrirait préférentiellement ainsi :

```
int add3(const int& a, const int& b){return a+b;}
```

Il existe d'autres usages du mot clé const. Certains seront abordés dans le module de C++ compléments.

2.11 Fonction assert

Le prototype de cette fonction est

```
void assert (int expression);
```

Si expression prend la valeur 0 (c'est à dire false), alors un message d'erreur est renvoyé et le programme est interrompu.

La fonction est désactivée si, au moment de l'inclusion <assert.h>, l'entité NDEBUG est définie selon la directive :

```
#define NDEBUG
```

2.12 Allocation dynamique

En C++, l'allocation dynamique se fait à l'aide des fonctions `new` et `delete`. La fonction `new` alloue de la mémoire et la rattache à un pointeur. La fonction `delete` libère la mémoire associée à un pointeur.

Ainsi, l'instruction

```
double* d=new double(5.1);
```

induit l'allocation dynamique de la mémoire nécessaire au stockage d'un `double`, et l'associe au pointeur `d`. Lorsque cet objet n'est plus utile, la mémoire est libérée par l'instruction

```
delete d;
```

De même, l'instruction

```
double* tabd=new double[10];
```

alloue dynamiquement de la mémoire nécessaire au stockage de dix `doubles`, et l'associe au pointeur `tabd`. Lorsque cet objet n'est plus utile, la mémoire est libérée par l'instruction

```
delete [] tabd;
```

En C++, comme en C, les indices commencent à 0 dans les tableaux : le pointeur de `double` défini par la dernière instruction `new` ci-dessus donne accès aux données `*tabd` à `*(tabd+9)`, ou de manière équivalente `tabd[0]` à `tabd[9]`.

2.13 Fonctions génériques

La notion de surcharge de fonction nous a permis de considérer les fonctions

```
int add(int a, int b){return a+b;}
double add(double a, double b){return a+b;}
```

Il existe une façon plus efficace de faire co-exister ces fonctions en utilisant la notion de fonction générique ("template"). Ici, on pourrait définir la fonction générique `add` suivante :

```
template<class K>
K add(K a, K b){return a+b;}
```

Si la fonction est définie de la sorte dans le header, alors elle est inline. Elle est considérée par le préprocesseur. Au moment où le préprocesseur rencontre l'appel de la fonction, il sait quel type doit jouer le rôle de `K`, et il sait comment remplacer l'appel de la fonction par son action avec le bon type. De ce fait, au moment où le compilateur rencontre l'action, le travail d'interprétation de la fonction générique est fait.

Il est aussi possible de déclarer la fonction dans un header :

```
template<class K>
K add(K a, K b);
```

et de définir la fonction dans un fichier .c++ :

```
template<class K>
K add(K a, K b){return a+b;}
```

Dans ce cas, au moment de la compilation du fichier .c++ qui contient la définition de la fonction, le compilateur ne connait pas les types avec lesquels la fonction sera utilisée. Il faut alors instantier la fonction, préciser les types avec lesquels elle devra être opérationnelle. Dans ce cas, le fichier .c++ contiendrait la définition et les instantiations :

```
template<class K>
K add(K a, K b){return a+b;}

template int add(int, int);
template float add(float, float);
template double add(double, double);
```

Une fonction générique peut dépendre de plusieurs types :

```
template<class K, class L>
K add(K a, L b){return a+b;}
```

Attention alors aux types avec lesquels cette fonction est utilisée. Que se passerait-il si elle prenait un premier argument de type `int` et un deuxième de type `double` ?

Par ailleurs, le compilateur ne fait pas de conversion : le modèle à un argument ne permettrait pas de faire l'addition d'un `int` avec un `double`.

Enfin, une fonction générique peut être spécifiée : la fonction générique précédente peut co-exister avec la déclaration et définition d'une fonction définie spécifiquement pour un type :

```
int add(int a, int b){return a+b;}
```

Dans ce cas, cette dernière est prise en compte prioritairement sur la fonction générique. On parle de spécialisation d'un `template`.

Chapitre 3

Les classes

3.1 Introduction

La notion de classe en C++ permet la programmation orientée objet. Elle permet de structurer le programme sur la base d'objets spécifiques définis à travers les classes. Une classe est un type défini par le développeur. La notion de classe est très riche. Nous abordons ici quelques aspects de cette notion.

La notion de classe est utilisée pour définir les objets d'un programme. Un objet est identifié par des données et des méthodes. Les données (données membres de la classe) sont représentatives de l'état courant d'un objet. Les méthodes (fonctions membres de la classe) sont les actions qu'il est possible de faire avec les objets de la classe.

3.2 Définition d'une classe

3.2.1 Définition

Une façon standard de définir une classe est la suivante :

```
class MaClasse{
private:
   // donnees membres
   int donnee1;
   double donnee2;
public:
   // constructeur (optionnel)
   MaClasse(...);
   // fonctions membres ou methodes
   int fonc1();
   double fonc2();
};
```

Une classe contient des données membres (des variables), et des fonctions membres, dont d'éventuels constructeurs et le destructeur. Un constructeur d'une classe porte le même nom que la classe. Le destructeur de la classe `MaClasse` porterait le nom `~MaClasse`. La définition proposée ci-dessus pourrait être contenue dans un fichier `MaClasse.h`. Les fonctions membres seraient alors définies dans un fichier `MaClasse.c++` qui contiendrait :

```
int MaClasse::fonc1(){...}
double MaClasse::fonc2(){...}
```

L'opérateur "`::`" est appelé opérateur de portée.

Les éléments de la classe `MaClasse` sont référencés à partir d'un objet de type `MaClasse`, par l'opérateur ".", et à partir d'un pointeur sur un objet de type `MaClasse`, par l'opérateur "->". Ainsi, nous pourrions définir

```
MaClasse objet1;
MaClasse* objet1_p(&objet1);
```

et désigner

```
objet1.donnee1
objet1_p->fonc2()
```

Créer un objet, c'est créer une instance de la classe. L'objet `objet1` est une instance de la classe `MaClasse`. On parle d'instantiation.

Il est d'usage de définir les données membres comme privées afin de les protéger, d'éviter qu'elles puissent être modifiées à l'extérieur de la classe. En effet, une donnée membre privée ne peut être référencée que dans la classe, notamment au sein des fonctions membres de la classe. Pour cela, le développeur utilise le mot clé `private`. Nous parlons alors d'encapsulation. Les fonctions membres sont publiques si elles doivent être référencées de l'extérieur de la classe. Des fonctions membres référencées uniquement au sein d'autres fonctions de la classe, sont préférentiellement privées.

3.2.2 Un exemple

Pendant plusieurs années, il n'y avait pas de type pour les nombres complexes en C++. De ce fait, les utilisateurs développaient leur propre classe de nombres complexes.

La définition d'une telle classe pourrait commencer ainsi, dans le fichier `Complexe.h` :

```
#ifndef COMPLEXE_H
#define COMPLEXE_H

class Complexe{
private:
   double re, im;
public:
   Complexe(double a, double b);
};
#endif  // COMPLEXE_H
```

et dans le fichier `Complexe.c++` :

```
#include "Complexe.h"

Complexe::Complexe(double a, double b){re = a; im=b;}
```

3.2.3 Donnée membre statique et fonction membre statique

On a vu qu'une classe contient des données qu'il convient autant que possible de définir privées. Ainsi, si nous définissons deux complexes

```
Complexe X,Y;
```

chacun d'eux possède sa propre donnée `re` et sa propre donnée `im`. Le développeur peut aussi définir des données membres statiques par le mot clé `static`. Cela induit que la donnée membre est partagée par tous les objets de la classe. Par exemple, dans la classe `Complexe`, nous pouvons déclarer la donnée membre statique suivante :

```
static Complexe eye;
```

Elle serait alors définie dans le fichier `Complexe.c++` ainsi, par exemple :

```
Complexe Complexe::eye(0., 1.);
```

Il est dans ce cas nécessaire qu'elle soit déclarée avec le mot clé `static` sans quoi un objet de la classe `Complexe` contiendrait un `Complexe` qui contiendrait un `Complexe` qui contiendrait un `Complexe` qui ... sans fin.

Des fonctions membres qui n'utilisent que des données membres statiques de la classe et ne dépendent pas des objets de la classe peuvent aussi être déclarées avec le mot clé `static`. Par exemple, dans la classe `Complexe`, nous pouvons déclarer la fonction membre :

```
static void afficheEye();
```

Il n'est pas nécessaire de déclarer cette fonction statique mais si nous le faisons, alors, nous pouvons invoquer la fonction sans passer par un objet de la classe. Un appel de la fonction dans une fonction appelant pourra être :

```
Complexe::afficheEye();
```

Le mot clé `static` n'est pas repris dans le fichier `Complexe.c++` lors de la définition de la fonction. Une fonction membre statique n'a pas accès aux données membres non statiques de la classe.

3.2.4 Pointeur `this`

Au sein d'une classe, chaque fonction membre non statique a accès à un pointeur sur l'objet courant, le pointeur `this`. Il s'agit d'un pointeur sur l'objet courant, de ce fait la quantité `*this` désigne l'objet courant. Cela est utile notamment pour la surcharge de l'opérateur = (cf sous-section 3.5.2). En effet, la surcharge de l'opérateur = se terminera par l'instruction `return *this;`. Il en sera de même pour toutes les fonctions et tous les opérateurs qui devront renvoyer une auto-référence (une référence sur l'objet courant).

Une fonction membre statique n'a pas accès au pointeur `this`.

3.3 Constructeurs, destructeur et opérateurs d'une classe

3.3.1 Constructeurs sans argument, par copie

Comme vu précédemment, une classe possède des constructeurs et un destructeur. Elle possède un constructeur par défaut sans argument d'entrée et un constructeur par copie qui sont implicites, fournis par le compilateur. Toutefois, le constructeur sans argument n'est plus disponible dès lors qu'un autre constructeur est défini dans la classe, à moins de redéfinir le constructeur sans argument explicitement. En C++11, elle possède aussi un constructeur implicite par "move". Le constructeur par move a été introduit en C++11 pour éviter la recopie induite par le constructeur par copie, lorsque cela est possible. Le constructeur par copie est utilisé :

- ⋆ lors d'une définition avec initialisation, par exemple :

  ```
  Complexe zz(z);
  Complexe zzz = z; // equivalent a : Complexe zzz(z);
  ```

  où `z` désigne un `Complexe` prédéfini. Dans le deuxième exemple, le caractère = n'induite pas une affectation mais bien un appel du constructeur par copie car il s'agit d'une déclaration.

- ⋆ lors du passage par valeur d'un objet à une fonction,

⋆ lorsqu'une fonction retourne un objet par valeur.

L'utilisateur peut récrire ces constructeurs implicites. Les prototypes sont visibles ici :

```
class MaClasse{
private:
   // donnees membres
public:
   MaClasse(); // constructeur par defaut
   MaClasse(const MaClasse&); // constructeur par copie
   MaClasse(MaClasse&&); // constructeur par move
};
```

Ce contenu se situerait dans le fichier `MaClasse.h`. La programmation de ces constructeurs dans le fichier `MaClasse.c++` serait ainsi :

```
MaClasse::MaClasse(){ ... }
MaClasse::MaClasse(const MaClasse& obj){ ... }
MaClasse::MaClasse(MaClasse&& obj){ ... }
```

3.3.2 Destructeur

Une classe possède également un destructeur par défaut qui peut être récrit avec le prototype suivant :

```
   ~MaClasse(); // destructeur
```

Notamment, lorsque la création d'un objet induit de l'allocation dynamique, il est logique de trouver la libération de la mémoire dans le destructeur.

On n'appelle pas explicitement le destructeur dans le programme, celui est appelé implicitement à la fin de la zone de portée de la variable objet correspondant.

3.3.3 Constructeurs avec valeurs d'initialisation, liste d'initialisation

D'autres constructeurs peuvent être prévus. Par exemple, dans la classe complexe de la sous-section 3.2.2, un constructeur prend deux arguments d'entrée :

```
   Complexe(double a, double b);
```

Ce constructeur peut alors prendre des arguments avec des valeurs par défaut, nous parlons de constructeur avec des valeurs d'initialisation. Par exemple :

```
   Complexe(double a=0, double b=0);
```

Dans ce cas, si le constructeur `Complexe` est appelé avec un seul argument, le deuxième est considéré avec la valeur par défaut de `b`, ici 0. Et si le constructeur `Complexe` est appelé avec zéro argument, alors le premier est considéré avec la valeur par défaut de `a`, également 0 ici.

Si tous les arguments ont une valeur d'initialisation, alors le constructeur par défaut est redéfini. Si seulement une partie des arguments a une valeur par défaut, ceux ayant une valeur par défaut doivent venir en dernier dans le prototype du constructeur.

Un autre aspect des constructeurs est la liste d'initialisation dont l'usage est conseillé pour l'attribution de valeurs ou construction des données membres. Ainsi, le constructeur de la sous-section 3.2.2 s'écrit préférentiellement :

```
Complexe::Complexe(double a, double b):re(a), im(b) {}
```

Lorsque certaines données membres sont des objets, la liste d'initialisation permet d'éviter une création des données membres par le constructeur par défaut suivie d'une affectation, et induit plutôt une création par le constructeur par copie.

Pour un constructeur avec davantage d'instructions, il est préférable de choisir une écriture plus lisible avec un passage à la ligne avant et après la liste d'initialisation :

```
Complexe::Complexe(double a, double b)
: re(a), im(b)
{}
```

3.3.4 Surcharge d'opérateurs

De manière générale, la surcharge d'opérateur comme fonction membre d'une classe se fait selon les prototypes suivants :

- ⋆ pour un opérateur unaire :

```
type operator op()
```

- ⋆ pour un opérateur binaire :

```
type1 operator op(type2)
```

Dans le cas d'un opérateur binaire, l'opérande de gauche est l'objet qui invoque l'opérateur et l'opérande de droite est l'objet en argument d'entrée.

Il y a plusieurs écritures possibles à l'usage des opérateurs :

operateur	fonction externe	fonction membre d'une classe
`op A` (unaire)	`operator op(A)`	`A.operator op()`
`A op B` (binaire)	`operator op(A,B)`	`A.operator op(B)`

3.3.5 Exemple (constructeurs, opérateurs)

Les sous-sections précédentes présentent différentes façon d'enrichir la liste de constructeurs d'une classe. Nous utilisons en fait la notion de surcharge d'une fonction. De la même façon, il est possible de surcharger les opérateurs. Par exemple, nous pouvons enrichir la classe `Complexe` présentée dans la sous-section 3.2.2 en récrivant le fichier `Complexe.h` ainsi :

```
#ifndef COMPLEXE_H
#define COMPLEXE_H

class Complexe{
private:
   double re, im;
public:
   Complexe(): re(0), im(0) {} //constructeur inline
   Complexe(double a, double b=0);
   void affiche();
   Complexe operator+(const Complexe& z);
   Complexe& operator+=(const Complexe& z);
};
#endif  // COMPLEXE_H
```

et le fichier `Complexe.c++` ainsi :

```
#include "Complexe.h"
#include <iostream>
using namespace std;

Complexe::Complexe(double a, double b):re(a), im(b) {}

void Complexe::affiche(){ cout<<"re = "<<re<<", im = "<<im<<endl; }


Complexe Complexe::operator+(const Complexe& z){
   Complexe zz(re+z.re, im+z.im);
   return zz;
}

Complexe& Complexe::operator+=(const Complexe& z){
   re+=z.re; im+=z.im;
   return *this;
}
```

Dans le cas de cet exemple, le constructeur à deux arguments d'entrée prend le deuxième argument avec une valeur d'initialisation. Le premier argument n'a pas de valeur d'initialisation. Ainsi ce constructeur peut co-exister avec le constructeur par défaut. Par ailleurs, les opérateurs "+" et "+=" sont surchargés.

3.3.6 Constructeur et mot clé `explicit`

L'exemple de la sous-section 3.3.5 contient le constructeur de prototype

```
   Complexe(double a, double b=0);
```

Il est donc possible de construire un complexe à partir d'un seul argument d'entrée de type double. De ce fait, si nous écrivons dans un programme :

```
   Complexe z(0,1.5);
   Complexe zz = z + 5.1;
```

alors, le compilateur fera appel au constructeur cité ci-dessus pour construire un complexe à partir du double 5.1 et fera une somme entre deux complexes. Il est possible d'interdire cet appel du constructeur en le définissant avec le mot clé `explicit`. Ainsi, le compilateur génèrera une erreur indiquant qu'il n'existe pas d'opération d'addition d'un complexe avec un réel. Pour définir le constructeur comme explicite, nous changeons son prototype dans le fichier `Complexe.h` de la sorte :

```
   explicit Complexe(double a, double b=0);
```

3.4 Mot clé `friend` et alternative

Dans la section 3.2, nous avons défini les données membres d'une classe comme privées. De ce fait, elles ne sont pas visibles depuis une fonction externe à la classe ou depuis une fonction membre d'une autre classe. Si nous souhaitons que les données membres soient visibles dans une telle fonction, il faut déclarer cette fonction comme "amie" au sein de la classe. Pour cela, le mot clé `friend` est utilisé. Cela a l'effet de rompre l'intérêt de l'encapsulation qui a pour objectif de protéger les données membres. Une citation donne le conseil suivant :

Tell your friends : “don’t use friend”

En effet, il est conseillé de n’utiliser le mot clé `friend` qu’en dernier recours lorsqu’aucune autre stratégie n’a été trouvée pour l’éviter. Notamment, pour l’éviter on peut avoir recours à des fonctions d’accès comme développé ci-après.

3.4.1 Prédéclaration d’une classe

Si le développeur souhaite définir une fonction de la classe C2, comme fonction “amie” de la classe C1, afin d’utiliser les données membres de la classe C1 dans cette fonction de la classe C2, il lui faut prédéclarer C1, définir C2, puis définir C1.

La prédéclaration de la classe C1 se fait ainsi :

```
class C1;
```

3.4.2 Fonction ou classe amie

Pour déclarer une fonction “amie” d’une classe C1, il faut que cette fonction soit définie classiquement, une fonction externe ou une fonction membre d’une autre classe déjà définie, puis elle est déclarée aussi dans la classe C1 avec le mot clé `friend` en début de son prototype :

```
friend type nomFonction(arguments d'entree);
```

Il est aussi possible de déclarer une classe comme amie d’une autre classe. Si on souhaite avoir accès aux données membres privées de la classe C1 dans la classe C2, on peut définir la classe C2 comme amie de la classe C1. Pour cela, on déclare C2 dans C1 avec le mot clé `friend` :

```
class C1{
private :
   int i ;
public :
   C1(int a=0);
   void affiche();
   friend class C2;
   // ou
   // friend void C2::affiche(C1);
};
```

Dans ce cas, toutes les fonctions membres de la classe C2 ont accès aux données membres de la classe C1. On pourrait limiter l’accès aux données membres de C1 à une fonction membre de C2 en déclarant cette fonction membre comme amie de la classe C1. Dans ce dernier cas, la déclaration de la classe C1 nécessite la déclaration préalable de la classe C2.

La classe C2 pourrait être de la forme :

```
class C2{
private :
   int j ;
public :
   C2(int a=0);
   void affiche();
   void affiche(C1);
};
```

avec dans le fichier `.c++` correspondant :

```
void C2::affiche(C1 c){
   cout<<"i␣from␣C1␣object␣=␣"<< c.i << endl;
}
```

La déclaration de la classe C2 nécessite la pré-déclaration de la classe C1.

3.4.3 Fonction d'accès

Une fonction d'accès à une donnée membre est une fonction qui renvoie la valeur de cette donnée membre. Par exemple, nous pouvons enrichir la classe `Complexe` de deux fonctions d'accès permettant de récupérer les parties réelle ou imaginaire d'un complexe :

```
#ifndef COMPLEXE_H
#define COMPLEXE_H

class Complexe{
private:
   double re, im;
public:
   Complexe(): re(0), im(0) {} //constructeur inline
   Complexe(double a, double b=0);
   double getRe(){return re;}
   double getIm(){return im;}
   void affiche();
   Complexe operator+(const Complexe& z);
};
#endif  // COMPLEXE_H
```

Vous constaterez qu'ici les fonctions d'accès `getRe` et `getIm` sont `inline` de manière implicite. Cela est souvent le cas pour les fonctions d'accès dont la programmation tient généralement en une ligne et une instruction.

3.4.4 Fonction déclarée `const`

Des fonctions membres d'une classe qui ne modifient pas l'objet qui les réfèrent peuvent être définies avec le mot clé `const` en fin de prototype. Par exemple, les fonctions d'accès et la fonction d'affichage de la classe `Complexe`, de la sous-section 3.4.3, devraient être définies ou déclarées de la sorte :

```
   double getRe() const {return re;}
   double getIm() const {return im;}
   void affiche() const;
```

3.5 Classes et allocation dynamique

Il est possible d'avoir recours à l'allocation dynamique dans une classe. Par exemple, si nous considérons une classe de vecteurs de complexes, nous considèrerons une classe dont une donnée membre devra stocker les valeurs des complexes. Pour cela, nous pouvons utiliser une donnée membre pointeur sur des complexes, initialisée par allocation dynamique, par un appel de `new`.

Une telle situation nécessite des précautions. En effet, afin d'éviter toute perte de mémoire, il faut libérer la place mémoire allouée dynamiquement, au moment de la destruction de l'objet. De ce fait, le destructeur de la classe fera appel à `delete`.

Mais alors, il faut prendre garde au fait que le constructeur par copie et l'affectation par copie par défaut s'opèrent de manière superficielle. La copie d'un objet A en un objet B, induit que la donnée

membre pointeur sur des complexes prend la même valeur pour les deux objets. De ce fait, la libération de la mémoire pour l'un induira l'impossibilité de la libération de la mémoire pour l'autre car celle-ci aura déjà été libérée.

Dans ce cas, il convient de prendre la précaution suivante : faire le choix soit de neutraliser les constructeurs et affectations par copie, soit de prévoir des constructeurs et affectations par copie en profondeur. Une copie en profondeur de A sur B consiste à allouer de la mémoire pour B et recopier les valeurs pointées par le pointeur de A à l'emplacement pointé par le pointeur de B.

3.5.1 Destructeur et `delete`

Lorsqu'un constructeur contient un `new` pour allouer de la mémoire dynamiquement pour une donnée membre de la classe :

```
MaClasse::MaClasse(int lo):lg(lo){
   val = new double[lg];
}
```

pour la classe :

```
class MaClasse{
private:
   int lg;
   double* val;
public:
   MaClasse(int);
};
```

alors, il convient que cette classe redéfinisse le destructeur :

```
class MaClasse{
private:
   int lg;
   double* val;
public:
   MaClasse(int);
   ~MaClasse();
};
```

avec l'implémentation :

```
MaClasse::~MaClasse(){
   delete [] val;
}
```

3.5.2 Constructeur et affectation par copie "superficielle" ou "en profondeur"

Pour la classe de la sous-section précédente, le constructeur par copie par défaut copie la valeur du pointeur `val` seulement. Il s'agit d'un constructeur dit par copie superficielle. Il en est de même pour l'affectation par défaut qui réalise une copie superficielle. Afin d'être compatible avec le destructeur qui libère la mémoire, le constructeur par copie et l'affectation par copie doivent être récrits de sorte à réaliser une copie en profondeur. Pour la classe `MaClasse`, ils seraient déclarés ainsi :

```
class MaClasse{
private:
   int lg;
   double* val;
```

```
public:
   MaClasse(int);
   MaClasse(const MaClasse&);
   ~MaClasse();
   MaClasse& operator=(const MaClasse&);
};
```

et seraient définis ainsi :

```
MaClasse::MaClasse(const MaClasse& MC){
   lg = MC.lg;
   val = new double [lg];
   for (int i=0; i<lg; i++){val[i] = MC.val[i];}
}

MaClasse& MaClasse::operator=(const MaClasse& MC) {
        if (lg != MC.lg) {
                delete [] val;
                lg = MC.lg; val = new double [lg];
        }
        for (int i=0; i<lg; i++) { val[i]=MC.val[i]; }
        return *this;
}
```

3.5.3 Neutralisation des constructeurs et affectations par copie ou move

La neutralisation des constructeurs et affectations par copie ou move se fait de la façon suivante dans le header pour la classe `MaClasse` :

```
   MaClasse(const MaClasse&) = delete;
   MaClasse(MaClasse&&) = delete;
   MaClasse& operator=(const MaClasse&) = delete;
   MaClasse& operator=(MaClasse&&) = delete;
```

3.5.4 Exemple

Un exemple d'une classe de vecteurs de complexes pourrait être celle-ci :

```
class VectC{
private:
   int lg;
   Complexe* val;
public:
   VectC(int lo=0);
   VectC(int lo, Complexe a);
   VectC(const VectC&);
   ~VectC();
   void affiche() const;
   VectC& operator=(const VectC&);
};
```

avec l'implémentation suivante :

```
VectC::VectC(int lo)
:lg(lo),val(new Complexe[lg])
{}
```

```
VectC::VectC(int lo, Complexe a)
:lg(lo),val(new Complexe[lg])
{
   for (int i=0; i<lg; i++){val[i] = a;}
}

VectC::VectC(const VectC& V)
:lg(V.lg),val(new Complexe[lg])
{
   for (int i=0; i<lg; i++){val[i] = V.val[i];}
}

VectC::~VectC(){
   delete [] val;
}

void VectC::affiche() const {
   for (int i=0; i<lg; i++){val[i].affiche();}
}

VectC& VectC::operator=(const VectC& V) {
        if (lg != V.lg) {
                delete [] val;
                lg = V.lg; val = new Complexe [lg];
        }
        for (int i=0; i<lg; i++) { val[i]=V.val[i]; }
        return *this;
}
```

3.6 Classe générique

Une classe peut aussi être définie générique avec le mot clé `template`, sous la forme suivante :

```
template<class C>
class MaClasseTemp {
private:
   // donnees membres
public:
   type fonctionCT(const C& data);
   // autres fonctions membres
};
```

Le type `C` est un argument. Il peut être utilisé pour déclarer une donnée membre ou dans une fonction membre. Le nom de type associé à cette classe générique est `template<class C> MaClasseTemp<C>`. Ainsi pour définir la fonction `fonctionCT` en dehors de la classe, on aurait :

```
template<class C>
type MaClasseTemp<C>::fonctionCT(const C& data){...}
```

Exemple :

```
template<class T>
class essai {
   T i;
};
```

Utilisation :

```
int main(){
   essai<int> ci;
   essai<float> cf;
}
```

Le compilateur fabrique les deux classes

```
class essai_i {int i;};
```

et

```
class essai_f {float i;};
```

Un exemple utile de classe générique est la classe `complex` de la bibliothèque standard. De la sorte, nous pouvons définir des complexes de `double` ou des complexes de `float`.

Pour plus de détails sur cette possibilité, nous renvoyons à [Str14] p665.

3.7 Classe base et classe dérivée

Nous abordons ici la notion de classe dérivée et d'héritage. Le C++ permet de définir une classe comme dérivée d'une autre classe. Elle hérite alors des données membres de la classe dont elle dérive, dite classe de base, ainsi que des fonctions membres. Cependant, les fonctions membres peuvent être récrites. Il s'agit du polymorphisme. Attention toutefois à ne pas utiliser le polymorphisme pour la construction de l'objet. En effet, tant que l'objet n'est pas construit, le polymorphisme n'est pas actif.

3.7.1 Définition

Si l'on souhaite définir une classe `MaClasseDer` dérivée de la classe `MaClasseBase`, on peut la définir ainsi :

```
class MaClasseDer : public MaClasseBase {
private:
   // donnees membres propres a cette classe
public:
   // fonctions membres de la classe
};
```

Un objet de la classe `MaClasseDer` est aussi un objet de la classe `MaClasseBase`.

3.7.2 Héritage

Dans la sous-section 3.7.1, le mot clé `public`, en première ligne de cette définition de classe, indique la façon dont la classe `MaClasseDer` hérite de la classe `MaClasseBase`. Dans ce cas, elle hérite des éléments de la classe de base protégés ou publics avec le même statut. Elle n'hérite pas des éléments membres privés. Les éléments membres protégés sont définis par le mot clé `protected` en place du mot clé `private` ou `public`. Ainsi, lorsqu'une classe est destinée à être classe de base, il est d'usage de déclarer ses données membres `protected`, et les fonctions membres `public` ou `protected` selon qu'elles sont référencées à l'extérieur de la classe ou pas.

La classe dérivée pourrait être définie avec un statut de dérivation différent de `public`. Cela impacterait la façon dont la classe dérivée hériterait des éléments de la classe de base. Voir [Str14] p605 pour plus de détails.

3.7.3 Constructeur

Parmi les fonctions membres de la classe, on trouve le constructeur. Celui-ci doit référer au constructeur de la classe de base, en liste d'initialisation du constructeur de la classe dérivée. Au sein de la définition de la classe dérivée, le constructeur s'écrirait :

```
MaClasseDer(...):MaClasseBase(...){...}
```

Ou alors, s'il est seulement déclaré dans la définition de la classe :

```
MaClasseDer(...);
```

l'implémentation de celui-ci se trouve dans le fichier .c++ correspondant :

```
MaClasseDer::MaClasseDer(...):MaClasseBase(...){...}
```

3.7.4 Fonction virtuelle

Afin d'avoir recours au polymorphisme, une fonction `MaFonction` définie dans la classe de base, que nous souhaiterions récrire dans la classe dérivée, doit être déclarée dans la classe de base comme virtuelle par le mot clé `virtual`. Elle est alors déclarée ainsi :

```
virtual type MaFonction(...);
// ou :
virtual type MaFonction(...) const ;
```

ou définie ainsi dans la définition de la classe :

```
virtual type MaFonction(...){...}
// ou :
virtual type MaFonction(...) const {...}
```

Il est également à noter que le constructeur d'une classe ne peut pas être virtuel. De même, le polymorphisme n'est pas applicable au moment de la création de l'objet. Les fonctions qui sont utilisées par le constructeur ne peuvent pas bénéficier du polymorphisme.

Par contre, le destructeur peut être virtuel, et doit l'être lorsqu'une classe dérivée possède des données membres allouées dynamiquement.

3.7.5 Fonction virtuelle pure et classe abstraite

Certaines fonctions doivent être nécessairement définies dans les classes dérivées d'une classe de base. Dans ce cas, une telle fonction de la classe de base doit être virtuelle pure. Sa déclaration se fait alors de la façon suivante dans la classe de base, où elle n'est généralement pas définie :

```
virtual type MaFonction(...) = 0 ;
// ou :
virtual type MaFonction(...) const = 0 ;
```

Une classe qui contient des fonctions virtuelles pures est une classe abstraite. Elle peut être référencée dans le programme pour des usages des objets de cette classe par contre, elle ne peut pas être référencée directement pour la création d'objets. Si elle possède des fonctions virtuelles pures, il existe des classes dérivées. Nous créons alors des objets de ces classes dérivées. Puis, lorsque nous les manipulons, nous pouvons les désigner comme des objets de la classe de base à l'aide de pointeurs sur des objets de la classe de base. Pour permettre cela, il est important que le destructeur de la classe de base soit virtuel afin que celui de la classe dérivée soit appelé.

3.7.6 Exemple

Nous proposons ici un exemple de classes `Cercle` et `Rectangle` dérivées de la classe de base `Geometry`.

Elles peuvent être définies dans le fichier `geometry.h`

```
#ifndef GEOMETRY_H
#define GEOMETRY_H

class Geometry{
protected:
   static const double pi;
public:
   virtual ~Geometry(){}
   virtual void affiche() const;
   virtual double surface() const = 0;
};

class Cercle: public Geometry {
private:
   double xCentre, yCentre, rayon;
public:
   Cercle(double xC, double yC, double r);
   void affiche() const;
   double surface() const {return pi*rayon*rayon;}
};

class Rectangle: public Geometry {
private:
   double longueur, largeur;
public:
   Rectangle(double Lo, double la);
   void affiche() const;
   double surface() const {return longueur*largeur;}
};

#endif  // GEOMETRY_H
```

avec l'implémentation dans le fichier `geometry.c++`

```
#include "geometry.h"
#include <cmath>
#include <iostream>
using namespace std;


const double Geometry::pi = 4.*atan(double(1.));


void Geometry::affiche() const {
   cout<<"Je␣suis␣Geometry,␣pi␣=␣"<<pi<<endl;
}


Cercle::Cercle(double xC, double yC, double r)
: Geometry(), xCentre(xC), yCentre(yC), rayon(r) {}
```

```
void Cercle::affiche() const {
   cout<<"Je␣suis␣Cercle,"<<endl;
   cout<<"␣␣␣␣␣␣xCentre␣=␣"<<xCentre<<endl;
   cout<<"␣␣␣␣␣␣yCentre␣=␣"<<yCentre<<endl;
   cout<<"␣␣␣␣␣␣␣␣rayon␣=␣"<<rayon<<endl;
   cout<<"␣␣␣␣␣␣surface␣=␣"<<surface()<<endl;
}

Rectangle::Rectangle(double Lo, double la)
: Geometry(), longueur(Lo), largeur(la) {}

void Rectangle::affiche() const {
   cout<<"Je␣suis␣Rectangle,"<<endl;
   cout<<"␣␣␣␣␣␣␣␣longueur␣=␣"<<longueur<<endl;
   cout<<"␣␣␣␣␣␣␣␣␣largeur␣=␣"<<largeur<<endl;
   cout<<"␣␣␣␣␣␣␣␣␣surface␣=␣"<<surface()<<endl;
}
```

On observe entre autres que la classe de base `Geometry` est une classe abstraite. Elle contient une fonction virtuelle pure, la fonction `surface`. La fonction `affiche` n'est pas virtuelle pure. On peut se passer de la définir dans la classe dérivée.

Dans cet exemple, vous pouvez aussi remarquer un autre usage du mot clé `const` pour définir ici une donnée membre statique constante, la donnée de π. Vous prendrez garde à la façon dont l'utilisateur doit définir cette constante dans le fichier .c++.

Un usage de ces classes peut être le suivant :

```
#include "geometry.h"

#include <iostream>
using namespace std;

int main (void)
{
//   Geometry Geo; // error: Geometry is an abstract class
   Cercle C(3,2,5);
   Rectangle R(5,4);

   C.affiche();
   R.affiche();

   Geometry* p = new Cercle(2,1,6); // possible if ~Geometry() virtual
   p->affiche();
   delete p;
}
```

Ici, on voit la construction d'un pointeur `p` sur un objet de type `Geometry` avec appel du constructeur de la classe dérivée `Cercle`. L'objet créé est bien de type `Cercle` mais il sera identifié par la fonction `main` comme un objet de la classe `Geometry`. C'est dans ce contexte que le destructeur de la classe `Geometry` doit être virtuel pour assurer l'appel du destructeur de la classe dérivée `Cercle` au moment de libérer le pointeur `p`.

Dans cet exemple, on a réuni la définition des trois classes et la définition des fonctions membres associées dans un seul fichier `.h` et un seul fichier `.c++`. On pourrait aussi créer les fichiers `rectangle.c++`

et `cercle.c++` pour contenir les définitions des fonctions membres des classes `Rectangle` et `Cercle`.

Il existe plusieurs façons de construire des classes dérivées, notamment à plusieurs niveaux. Des précautions sont à prendre lorsque les schémas de dérivation font intervenir des diamants. C'est le cas si la classe D dérive des classes B1 et B2 lorsque B1 et B2 dérivent toutes deux d'une même classe B. Dans ce cas, il faudra que la classe de base soit virtuelle, et la déclaration de B1 et B2 se fera ainsi :

```
class B1:virtual public B{...};
class B2:virtual public B{...};
class D:public B1, public B2{...};
```

Cela évite que D contienne deux exemplaires de B.

Bibliographie

[DLss] Eric Darrigrand and Yvon Lafranche. FastMMLib documentation, IRMAR – Université de Rennes, in progress.

[Str14] Bjarne Stroustrup. *The C++ Programming Language, Fourth Edition*. Addison Wesley, 2014.